

MILESTONE 1 DOCUMENTATION

1. FHIR Validation Endpoint

1.1 Overview of FHIR Validation

The FHIR Validation Endpoint is the first major component of the MEDISPHERE Healthcare Management Platform. Its primary responsibility is to validate incoming FHIR Patient resources before they are persisted in the healthcare database. In a healthcare environment, patient information must maintain a consistent structure and must satisfy predefined validation rules before it can be used by other parts of the system. Invalid or incomplete patient information can negatively affect subsequent healthcare operations and can result in inconsistent records. Therefore, the FHIR validation component acts as an important entry point for maintaining patient-data quality and integrity.

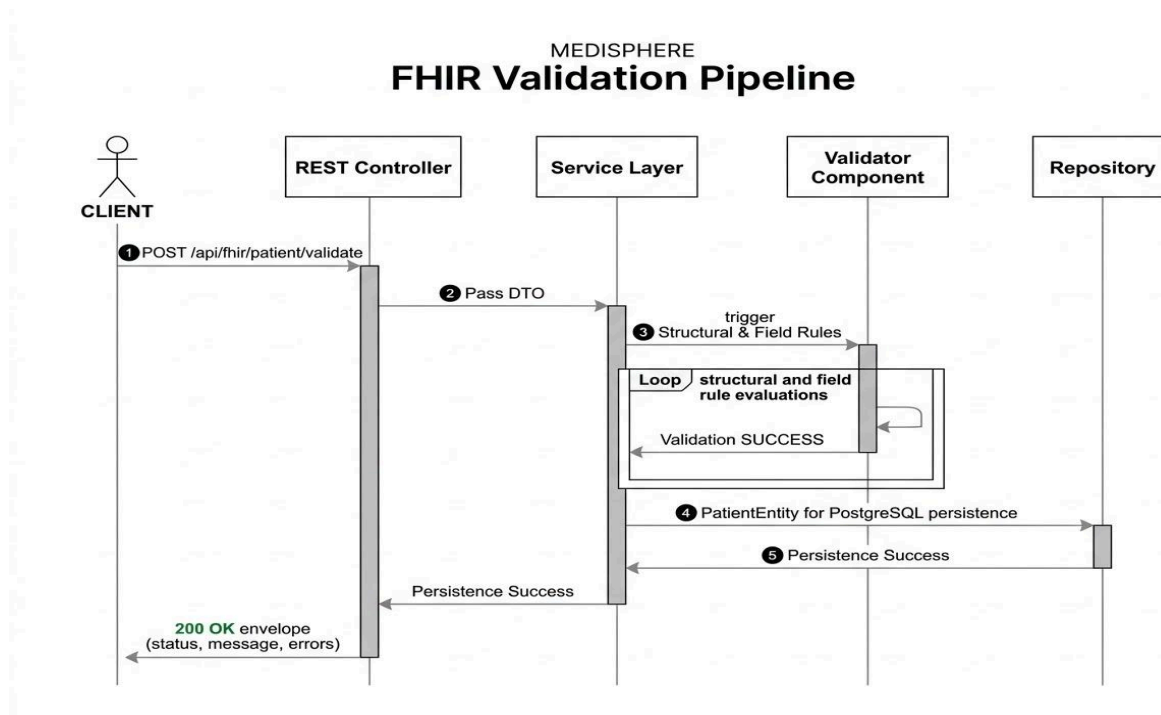
The project documentation defines the FHIR validation endpoint as a POST API exposed through [/api/fhir/patient/validate](#). The endpoint accepts a Patient resource and performs validation before allowing the information to be persisted. The documented validation rules include checking that the **resourceType** is present, ensuring that the patient ID is provided, restricting gender to the supported values, and validating the birth date according to the specified format and logical constraints.

1.2 Role of the FHIR Endpoint in MEDISPHERE

The FHIR endpoint functions as a boundary between external healthcare data and the internal MEDISPHERE persistence layer. Instead of allowing an incoming JSON request to directly access the database, the request passes through the application's validation and business-logic layers. This approach ensures that the data stored in the database has already passed the validation rules defined by the application.

The overall processing can be represented as a controlled validation pipeline where the client submits a Patient resource, the REST controller receives the request, validation is performed, business rules are evaluated, and only a valid resource proceeds toward persistence. If validation fails, the system returns a failure response instead of storing the invalid record. This separation is important because it prevents invalid data from propagating into downstream healthcare workflows.

Architecture Diagram: FHIR Validation Pipeline



1.3 REST API Design

The FHIR module exposes the following endpoint:

Endpoint: `/api/fhir/patient/validate`

HTTP Method: `POST`

The POST method is appropriate because the client is submitting a Patient resource for processing. The request body contains the patient information in a structured JSON representation. After processing, the endpoint returns a consistent response containing a status, message, and errors. The project documentation specifically defines a common response envelope consisting of `status`, `message`, and `errors` for the major APIs.

The use of a consistent response contract is important for frontend applications and other services consuming the API. Instead of requiring every client to interpret a completely different response structure, the client can always check the status, read the message, and inspect the errors collection when validation fails. This creates a predictable communication contract between the backend and its consumers.

1.4 Patient Resource Validation

The Patient resource contains several pieces of information that are important for identifying and describing a patient. The validation process checks the required attributes before persistence. According to the project documentation, the main validation fields are `resourceType`, patient ID, gender, and birth date.

The **resourceType** field identifies the type of FHIR resource being submitted. Since this endpoint is specifically intended for Patient resources, the service must ensure that the incoming request represents a Patient resource. This prevents an incorrectly structured resource from being processed as a patient record.

The patient identifier is another mandatory field. A healthcare system needs a reliable identifier to reference a patient across different operations. The project also defines the patient identifier as unique in the database design, providing an additional mechanism for preventing duplicate patient records.

1.5 Gender Validation

Gender validation is implemented as a controlled-value validation rule. The system enforces strict adherence to supported enumeration values to ensure interoperability across health records.

male – Patient identifies as male.

female – Patient identifies as female.

other – Patient identifies as another gender category.

unknown – Gender is unknown or unspecified.

This type of validation is important for maintaining data consistency. If arbitrary values were accepted, different applications could store the same conceptual information using different strings. Such inconsistency would make filtering, reporting, analytics, and interoperability more difficult. Restricting the field to predefined values creates a predictable data model.

1.6 Birth Date Validation

The birth date is validated using two important rules. First, it must follow the **yyyy-MM-dd** format. Second, it cannot represent a future date. These rules prevent both syntactical errors and logically invalid patient information.

Date validation is particularly important because dates can easily create subtle data-quality problems. A value may appear to be a date but still be incorrectly formatted or impossible in the context of a patient's birth date. By applying both format validation and future-date validation, the system ensures that the stored value satisfies both structural and business requirements.

1.7 Validation and Business Logic Separation

The MEDISPHERE architecture separates HTTP request handling from business validation. The documentation describes the architecture as stateless REST controllers with Bean Validation at the DTO boundary and business validation in the service or validator layer. The general structure is Controller » Service » Validator » Repository/Entity.

This separation is technically significant because controllers should primarily be responsible for receiving HTTP requests, mapping request data, and returning HTTP responses. The actual business rules belong in service or validator components. This makes the validation logic easier to unit test and modify without changing the API layer.

For example, if a future requirement changes the accepted gender values or introduces additional birth-date rules, the corresponding validation logic can be modified without rewriting the complete REST controller.

1.8 Persistence of Validated Patients

After a Patient resource passes validation, the resource can be persisted into the shared PostgreSQL database. The documentation explicitly states that validated patients are persisted through **PatientEntity**.

The database design identifies the **patients** table with fields including **dbId**, **patientId**, **resourceType**, **active**, **gender**, and **birthDate**. The patient ID is identified as unique, which supports the prevention of duplicate patient records.

The persistence mechanism uses Spring Data JPA and Hibernate ORM. This allows the Java entity representation of a patient to be mapped to a relational database table. Instead of manually constructing SQL statements for every operation, the application uses the repository abstraction provided by Spring Data JPA.

1.9 FHIR Validation Error Handling

The FHIR endpoint must also provide meaningful feedback when validation fails. The project uses a consistent response format containing a status, message, and errors. The API documentation identifies **200 OK** as the successful response, **400 Bad Request** for validation or business failures, and **500** for unexpected errors.

This approach allows a client application to distinguish between a successfully processed patient and a patient that failed validation. The errors collection is particularly useful because it can communicate the specific validation problems that need to be corrected.

1.10 Technical Importance

The FHIR validation component establishes the data-integrity layer of MEDISPHERE. It ensures that patient resources are not treated as valid merely because they can be parsed as JSON. Instead, the system applies explicit healthcare-data rules before persistence. This creates a stronger foundation for the remaining components because consent verification and auditing can operate using patient records that have already passed the initial validation process.

2. Consent Verification Endpoint

2.1 Overview of Consent Verification

The second major component of Milestone 1 is the Patient Consent Verification Endpoint. Its purpose is to verify whether the submitted patient consent is valid and approved for a particular clinical action. The endpoint is:

Endpoint: `/api/consent/verify`

HTTP Method: `POST`

The project identifies four supported consent types: `SURGERY`, `TREATMENT`, `DATA_SHARING`, and `EMERGENCY`. The consent verification component therefore introduces an authorization-oriented business layer into the healthcare platform.

Consent is important because healthcare actions may involve sensitive information or significant clinical decisions. The system should therefore verify that the required consent conditions are satisfied before considering a clinical action authorized.

2.2 Consent Request Processing

The consent endpoint receives a structured request containing `patientId`, `consentType`, `consentStatus`, and `authorizedBy`. These fields are explicitly identified in the project's consent validation rules.

The request first passes through the REST controller and request validation layer. Once the basic structure has been validated, the service layer performs the business-level consent checks. This separation allows the system to distinguish between an incorrectly constructed request and a correctly structured request that fails a business rule.

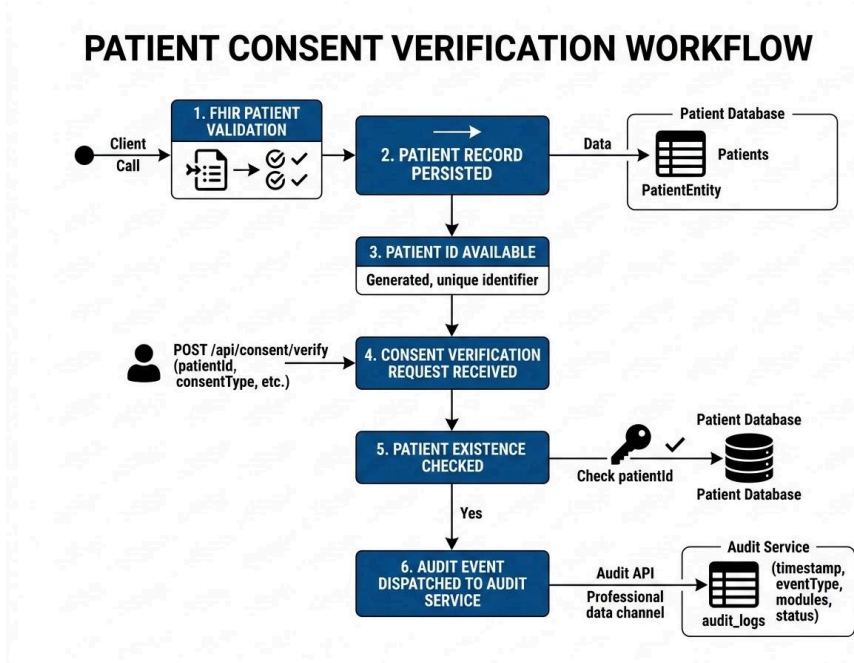
2.3 Patient Identification

The `patientId` field is mandatory because consent must be associated with a specific patient. The system needs to determine which patient the consent applies to before it can process the request.

The relationship between the FHIR and Consent components is therefore important. The FHIR component creates and persists validated patient records, while the Consent component can verify whether the referenced patient exists. This creates a logical dependency between the two workflows while keeping the services themselves separated.

The relationship between the FHIR and Consent components is critical to ensure patient data availability before authorization checks occur.

Workflow Diagram: Integrated Consent Verification Flow



The conceptual sequence coordinates validated patient persistence with downstream verification and audit logging.

2.4 Consent Type Validation

The consent service restricts consent types to four predefined clinical categories. The table below outlines the supported consent types and their functional scopes:

Consent Type	Functional Description
SURGERY	Authorizes surgical procedures and associated operative risks.
TREATMENT	Authorizes general clinical treatments, therapeutic protocols, and medication regimes.
DATA_SHARING	Authorizes transmission of PHI (Protected Health Information) to external entities or research networks.
EMERGENCY	Overrides standard authorization delays during critical care and life-threatening emergencies.

The purpose of this restriction is to ensure that consent requests are associated with recognized clinical actions. If arbitrary strings were accepted, the system could end up with inconsistent representations of the same type of clinical activity.

The controlled-value approach also makes the system easier to maintain. Other components can rely on a known set of consent types rather than having to handle unlimited user-defined values.

2.5 Consent Status Validation

One of the central rules of the Consent component is that `consentStatus` must be true. The documentation explicitly states this requirement.

This means that the verification process is not simply checking whether a consent record exists in the request. It evaluates whether the submitted consent is actually marked as approved. A false consent status therefore results in a failed verification decision.

The business logic can be expressed as:

2.6 Authorized By Validation

The `authorizedBy` field identifies the authority associated with the consent request. The documentation specifies that this field is required.

Including this information provides additional context about the consent decision. In a healthcare system, it is useful to know not only which patient and action were involved but also which authorized party is associated with the decision.

2.7 Stateless Consent Verification

An important characteristic of the current Consent component is that it is stateless. The project documentation explicitly states that the current state is “Stateless by design” and that the persistence layer is planned.

This means that the current service verifies the consent request that it receives but does not yet maintain a dedicated persistent consent record.

The planned database design includes a `consents` table containing:

However, this persistence layer is identified as planned rather than currently implemented in the project documentation.

2.8 Patient Existence Verification

Consent verification is also associated with patient existence. The service can verify whether the referenced patient is available in the patient database before accepting the consent decision.

This provides an important integrity check. A consent request containing a patient ID that does not correspond to a known patient should not be treated as a valid healthcare consent operation.

2.9 Exception Handling

The Consent component uses centralized exception handling so that validation and business errors can be converted into standardized API responses. This is important because multiple validation conditions can occur at different stages of request processing.

For example, a missing required field represents a request validation problem, whereas an unsupported consent type represents a business validation problem. Both need to be communicated to the client in a predictable manner.

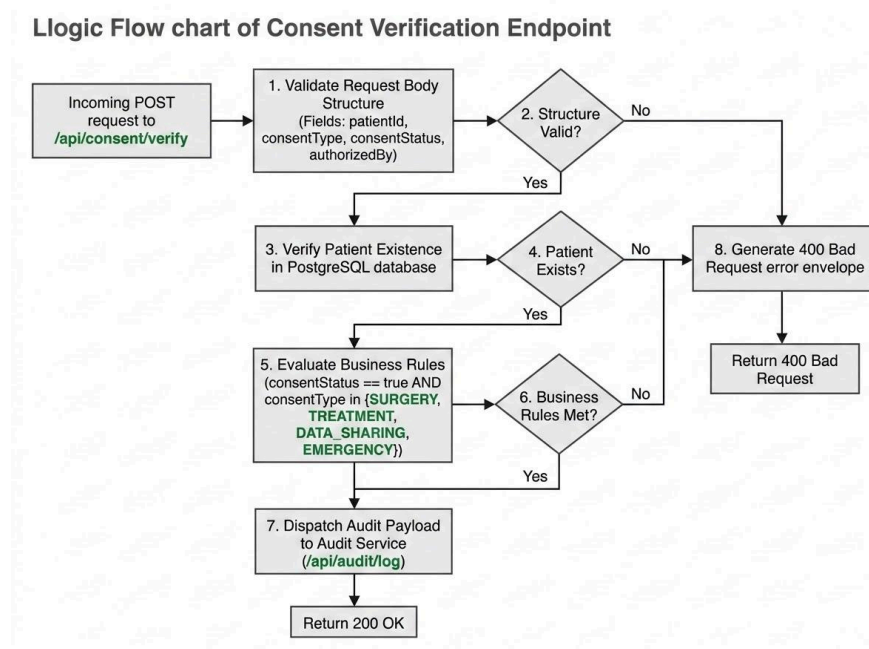
The common API response contract described by the project provides the **status**, **message**, and **errors** structure.

2.10 Consent Verification and Audit

A particularly important relationship exists between the Consent and Audit components. A consent decision should not simply disappear after the API returns its response. The decision is also sent to the Audit Service so that the system can maintain a persistent record of the event.

Therefore:

Diagram: Consent Verification Decision & Audit Dispatch Flow



This relationship is one of the main reasons why the four components of Milestone 1 should be understood as one integrated architecture rather than four independent features.

2.11 Technical Importance

The Consent Verification component provides the authorization layer of MEDISPHERE. While FHIR validation answers whether the patient data is valid, consent verification addresses whether a particular clinical action has the required approval. The separation of these responsibilities allows each business concern to evolve independently while still participating in a common healthcare workflow.

3. HIPAA Audit Log Persistence

3.1 Overview of Audit Logging

The third major component is HIPAA Audit Log Persistence. The purpose of this component is to maintain a persistent chronological record of important system decisions, particularly FHIR validation and consent-verification events.

The project documentation describes the purpose as creating an immutable, chronological record of consent-verification and validation events. It also identifies audit data such as timestamp, user information, action, patient information, resource type, IP address, status, and details.

Audit logging is essential in healthcare software because it provides traceability. When a sensitive operation occurs, the system should have a record that can later be examined to understand what happened.

3.2 Purpose of an Audit Trail

An audit trail allows the system to answer questions such as which operation occurred, which patient or resource was involved, whether the operation succeeded or failed, and when the event occurred.

For example, suppose a consent verification fails. The API response informs the current client about the failure, but the audit record provides historical evidence that the verification occurred.

Therefore, there are two different outputs:

This distinction is fundamental to audit architecture.

3.3 Dedicated Audit Service

MEDISPHERE implements auditing as an independent service rather than placing audit persistence directly inside every other module. This is consistent with the project's service-per-concern architecture.

The FHIR and Consent services communicate with the Audit Service through an API. The Audit Service then handles the persistence of the event.

The architecture can therefore be represented as:

This prevents the FHIR and Consent services from becoming tightly coupled to the internal implementation of the audit database.

3.4 Audit API

The Audit Service provides an API for creating audit records. The main operation is:

The service also provides retrieval operations for obtaining audit records, including retrieving the collection of logs and individual logs.

This makes the Audit Service more than a simple internal logging utility. It functions as an independent backend component responsible for the lifecycle of audit information.

3.5 Audit Event Structure

The current implementation represents an audit event using information such as event type, module, resource type, resource ID, action, status, message, and timestamp.

A FHIR validation event can therefore conceptually represent:

A consent event can similarly represent:

This structure makes the audit record understandable without requiring access to the original API request.

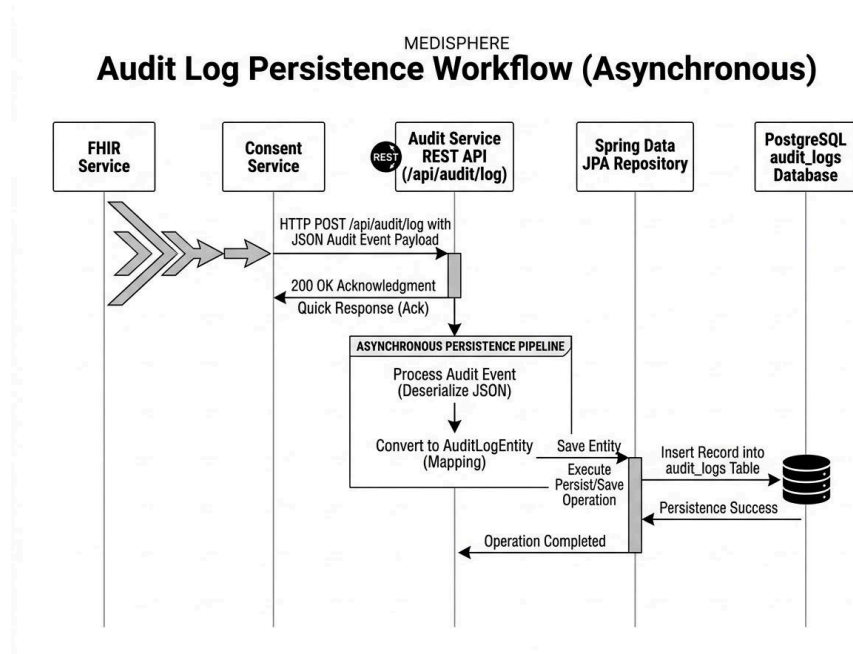
3.6 Audit Persistence Layer

The Audit Service uses a persistence layer based on Spring Data JPA and Hibernate. The audit event is represented as a Java entity, and the repository is responsible for interacting with the database.

The general architecture is:

This follows the same layered architecture used by the other MEDISPHERE services.

Architecture Diagram: Cross-Service Audit Logging Pipeline



3.7 Audit Database

The audit records are stored in the `audit_logs` table. The project documentation identifies the audit table as containing fields such as ID, timestamp, user name, action, patient ID, resource type, IP address, status, and details.

However, there is an important implementation distinction. The current implementation does not contain every field listed in the broader audit specification. Therefore, the documentation should distinguish between the target audit model and the currently implemented audit entity.

The current implementation is centered around event classification, module, resource information, action, status, message, and timestamp. The actor/user-name and source-IP requirements should be treated as enhancements unless they are added to the implementation.

3.8 Chronological Audit Records

Timestamp information allows audit records to be ordered chronologically. This is important because the sequence of events can be as important as the individual events themselves.

For example:

A chronological record allows administrators and auditors to reconstruct the sequence of activities associated with a patient or healthcare operation.

3.9 Audit Retrieval

The Audit Service provides APIs for retrieving audit records. This makes it possible to inspect previously recorded events rather than treating audit logs as write-only information.

Retrieval functionality can support activities such as reviewing events for a particular module, examining successful or failed operations, and inspecting a particular audit record.

The project documentation identifies the audit service as a component responsible for persistent logging and indicates that it is already wired into the consent-verification flow, with extension to FHIR validation planned in the documented state.

3.10 Cross-Service Audit Communication

The FHIR and Consent services use a client component to communicate with the Audit Service. Instead of directly inserting records into the `audit_logs` table, they send an audit request to the dedicated Audit API.

This is an important microservice principle:

The former keeps service boundaries cleaner and allows the Audit Service to control its own persistence logic.

3.11 Audit Failure Consideration

Because audit logging is performed through service-to-service communication, network or service failures can affect audit delivery. A production-grade healthcare platform would need to consider mechanisms such as retry policies, durable queues, transactional outbox patterns, or other guaranteed-delivery strategies.

The current project should not claim those mechanisms unless they are actually implemented. For Milestone 1 documentation, it is more accurate to describe the current REST-based audit integration and identify guaranteed audit delivery as a future hardening area.

3.12 Immutability Consideration

The documentation uses the term immutable for the audit record.

However, persistent storage by itself does not guarantee strong immutability. A production system would need additional technical controls to prevent unauthorized modification or deletion of audit records.

Possible future mechanisms include append-only storage, restricted database permissions, database-level update/delete restrictions, cryptographic integrity mechanisms, or specialized immutable storage.

Therefore, a technically accurate documentation statement would be:

The current implementation provides persistent chronological audit records, while stronger tamper-evident and immutable storage mechanisms can be introduced as a future security enhancement.

4. Independent Deployability and Testability

4.1 Overview

The fourth major component is Independent Deployability and Testability. Unlike the first three components, this is an architectural capability rather than a single endpoint.

The project explicitly defines the objective as keeping each module independently deployable and testable. The overall project architecture consists of separate backend concerns for FHIR validation, patient consent verification, and audit logging.

The reason for this architecture is to reduce the complexity associated with a single monolithic application.

4.2 Service-per-Concern Architecture

The project follows a service-per-concern principle. The documentation states that each service owns exactly one domain concern.

The responsibilities are divided as follows:

This separation ensures that business rules remain within the service that owns them.

4.3 Why Separation Is Required

The project documentation identifies limitations of monolithic services, explaining that monolithic services are harder to test, reason about, and evolve as regulations change.

Healthcare systems are particularly likely to experience changing requirements because validation rules, consent policies, security requirements, and compliance requirements can evolve over time.

With separated services, a change in one concern does not automatically require rewriting the entire backend.

For example, if FHIR validation rules change, the FHIR service can be modified independently from the internal implementation of the Audit Service.

4.4 Independent REST APIs

Each service exposes its own REST API.

The project documentation states that each module has its own REST API and is independently deployable.

This means that services interact through clearly defined interfaces rather than through direct access to each other's internal classes.

For example:

This provides clear service boundaries.

4.5 Independent Application Lifecycle

Each Spring Boot service has its own application lifecycle. This means the FHIR service can be started, stopped, built, tested, and deployed without requiring the entire MEDISPHERE backend to be packaged as one application.

The same principle applies to Consent and Audit.

Conceptually:

This provides greater flexibility than a monolithic deployment model.

4.6 Independent Testing

Independent deployment is closely connected to independent testing. Because each service owns a specific domain, unit tests can focus on that domain without requiring the entire platform.

FHIR tests can focus on:

This makes failures easier to isolate.

4.7 Layered Architecture Within Each Service

Independence exists not only between services but also within each service.

The project defines the general structure as:

The Controller handles HTTP communication, the Service handles business operations, the Validator handles business rules where applicable, and the Repository/Entity layer handles persistence.

This architecture provides two levels of separation:

This is what makes the overall system modular.

4.8 Stateless REST Controllers

The project specifically describes the REST controllers as stateless.

Statelessness means that the server does not need to maintain conversational session state between requests for these APIs. Each request contains the information required to process that operation.

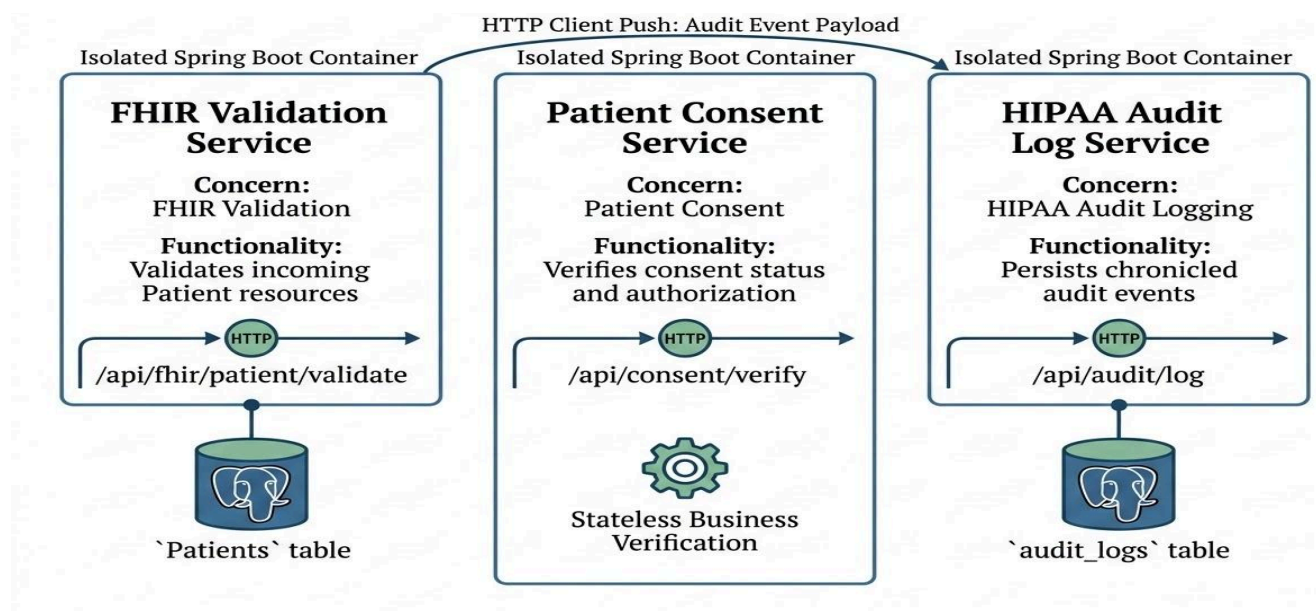
This simplifies deployment because any available instance of the service can process a request without depending on a particular server-side session.

4.9 Containerization and Deployment

Independent services can also be packaged and deployed separately using containerization.

The architecture can be represented as:

Architecture Diagram: Service-per-Concern Microservice Ecosystem



Each container contains the corresponding Spring Boot application and its runtime environment.

This approach makes deployment more consistent across development, testing, and production environments.

4.10 Independent Scaling

A further architectural advantage of independent services is that services can eventually be scaled according to their workload.

For example, if audit traffic becomes significantly higher than FHIR traffic, the Audit Service could be scaled independently. Similarly, if FHIR validation receives a large number of requests, additional FHIR service instances can be deployed without necessarily scaling the other services.

This is an architectural advantage of the design; it should not be interpreted as an assertion that autoscaling is already configured in Milestone 1.

4.11 Fault Isolation

Service separation also creates a degree of fault isolation.

For example:

The failure of one service does not inherently require all services to terminate. However, because FHIR and Consent communicate with Audit for audit events, production deployment should include a clear strategy for handling temporary audit-service failures.

This could eventually involve retry mechanisms, queues, circuit breakers, or durable event storage.

4.12 Maintainability

Independent deployability improves maintainability because developers can work on a specific business concern without having to understand the entire platform.

For example, developers working on FHIR validation primarily need to understand the Patient resource model, validation rules, persistence, and the Audit API integration. They do not need to modify the internal implementation of the Consent or Audit services for every FHIR change.

This is particularly useful for team-based development because different developers can own different services.

4.13 Technology Foundation

The project uses Java 25 and Spring Boot 4.1.0, with Spring MVC for the web layer and Spring Data JPA/Hibernate ORM for persistence. PostgreSQL 16 is used as the database and Maven is used as the build tool.

These technologies provide the foundation for implementing the independently deployable services.

5. Service Specifications & Architecture Summary

The platform features three independent REST services operating cohesively. The API Endpoints, Consent Types, and Database Schema mapping are summarized below.

5.1 API Endpoints Summary

Endpoint URL	HTTP Method	Description
/api/fhir/patient/validate	POST	Validates patient structure, ID, gender, birth date and persists to DB.
/api/consent/verify	POST	Verifies patient consent status, authorization entity, and patient existence.
/api/audit/log	POST	Persists HIPAA compliance audit records dispatched by other microservices.

5.2 Database Schema Overview

Service	Table Name	Key Columns & Schema
FHIR Service	patients	db_id (PK), patient_id (UNIQUE), resource_type, active, gender, birth_date
Consent Service	consents (Planned)	id (PK), patient_id (FK), consent_type, consent_status, authorized_by
Audit Service	audit_logs	id (PK), timestamp, event_type, module, resource_type, resource_id, action, status, message

6. Technical Significance of Milestone 1

Milestone 1 provides the foundation for the MEDISPHERE platform by addressing three important healthcare-system requirements: data integrity, consent, and traceability. FHIR validation prevents malformed or logically invalid Patient information from entering the persistence layer. Consent verification ensures that clinical actions are associated with an approved consent status. Audit logging creates a persistent history of important validation and consent decisions. Independent deployment separates these concerns into services that can be developed, tested, and deployed independently.

The architecture also provides a clear separation between API handling, business logic, validation, and persistence. This makes the system easier to maintain and provides a stronger foundation for future extensions.